



Progettazione di applicazioni web full stack

Introduzione al corso



Prof. Devis Bianchini

Dip. di Università degli Studi di Brescia



Informazioni generali

Docente: Devis Bianchini
Dipartimento di Ingegneria
dell'Informazione (DII, via
Branze, 38, I piano)

- Ufficio 31
- devis.bianchini@unibs.it

Home page del corso:
Progettazione di
applicazioni web full stack
(a.a. 2025)

- Uso della piattaforma
Moodle

Orario del corso:
Lunedì 13–15 (MLAB2)
Martedì 13–16 (MLAB2)
***Orario di ricevimento del
docente:***
Mercoledì 10–12



Architetture web avanzate

- Evoluzione delle architetture web e delle architetture di riferimento
- Architetture modulari e componenti
- Separazione tra frontend e backend
- Aspetti qualitativi delle applicazioni web complesse

Tecnologie e strumenti per lo sviluppo web avanzato

- Introduzione al «*build system*» Nx
- Concetti di monorepo, organizzazione del progetto e gestione delle dipendenze



Sviluppo frontend e backend

- Il linguaggio *TypeScript*
- Progettazione e implementazione del backend con *Node.js* (e *NestJS*)
- Sviluppo di applicazioni web frontend basate su architetture a componenti (*React.js*)

Progettazione model-driven delle applicazioni web

- Approcci model-driven per lo sviluppo web
- Il framework di modellazione WebML (Web Modeling Language)
- Modellazione dei dati, dell'ipertesto e dei componenti frontend



Libri di testo (consigliati per approfondimenti)

- Frank Zammetti, «Modern Full-Stack Development: Using TypeScript, React, Node.js, Webpack, Python, Django, and Docker», 2nd Edition, Apress Ed., 2022
- Andrew Mead, «Learning Node.js Development», Packt Publishing Ed., 2018
- Alex Banks, Eve Porcello, «Learning React: Modern Patterns for Developing React Apps», 4a edizione, O'Reilly Media, 2020.
- Jeffrey Zeldman, «Designing with Web Standards», Third Edition, New Riders Ed., 2009
- Christopher D. Hundhausen, «Full-stack Web Development from the Ground Up: Principles, Practices, and Technologies», First Edition, Morgan Kaufmann Pub, 2026

I lucidi e gli esercizi del corso costituiscono parte integrante del materiale didattico necessario alla preparazione dell'esame



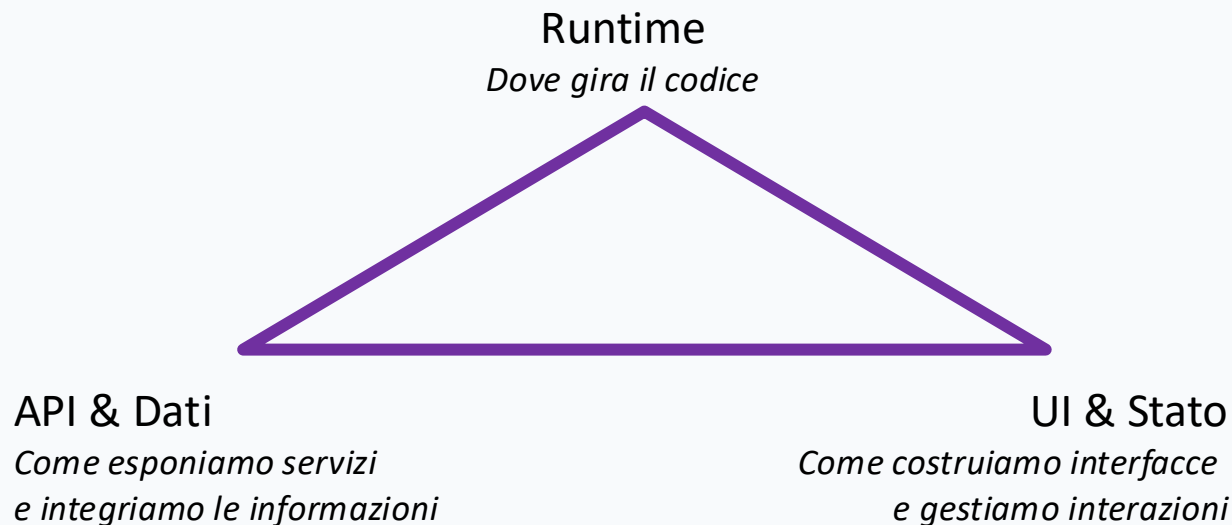
- La modalità d'esame consiste in
 - una prova scritta (più semplice) svolta in aula dallo studente, al calcolatore
 - una prova di approfondimento (più complessa), costituita da un elaborato sviluppato dallo studente, su specifiche concordate con il docente
 - una prova orale, in cui verranno discussi l'elaborato sviluppato dallo studente e la prova scritta

La prova di approfondimento va presentata in concomitanza con la prova scritta

La storia dello sviluppo web



- Le grandi «*svolte*» nello sviluppo di applicazioni web arrivano quando cambia uno di questi 3 pilastri:





- Da dove veniamo, dove siamo, dove andiamo
 - Passato: server-rendering, applicazioni monolitiche, prime soluzioni basate sul pattern MVC
 - Presente: API + frontend ricchi, cloud, componenti
 - Futuro: full-stack più integrato, ottimizzazione UX
- Tre protagonisti «principali»:
 - **TypeScript** → scalare lo sviluppo con sicurezza
 - **Node.js (NestJS)** → runtime + piattaforma per backend e tooling (ecosistema di sviluppo)
 - **React** → UI a componenti (composizione, riuso, architetture moderne)



1965 – Due computer al MIT Lincoln Lab comunicano tra loro scambiandosi ***pacchetti*** – *le comunicazioni diventano più robuste e scalabili*

1969 – Viene sviluppato ARPANET, il predecessore di Internet – *si collegano computer diversi, per scambiare dati e risorse*

1983 – ARPANET e Defence Data Network (DDN) iniziano ad usare il ***protocollo TCP/IP***; nasce Internet («rete di reti») – *un linguaggio comune per comunicare*

1984 – La Finlandia avvia FUNET (Finnish University Network), una rete pensata per università e ricerca – *agganciarsi rapidamente alle reti internazionali*

1988 – La Finlandia ottiene accesso ad ARPANET; il finlandese Jarkko Oikarinen crea IRC, un ***sistema di chat in tempo reale*** – *connettività significa prime applicazioni per collegare persone*

Dai «documenti» alle «applicazioni»



- La prima «svolta»: web statico → dinamico server-side → pressione verso il client
 - Web di «documenti»: pagine statiche, navigazione ipertestuale
 - Dinamicità lato server: CGI/PHP (e simili) generano HTML on-demand
- Un modello che funziona ... fino ad un certo punto
 - Semplice, perché il server controlla tutto
 - Limite UX: ogni interazione significativa prevede un ricarico della pagina (postback)

DOM + JavaScript: interazione dinamica



- Il browser non è più solo un «visualizzatore di pagine», ma una *piattaforma programmabile*:
 - **DOM (Document Object Model)** – modello della pagina
 - **JavaScript** – Logica di interazione
- Cosa abilita il DOM: manipolazione di pagine runtime
 - Cambiare elementi, contenuti, stili
 - Interazioni più ricche senza ricostruire tutta la pagina
- JavaScript: da «animazioni» a software
 - Per anni percepito come un linguaggio «per piccole cose»
 - L'UI moderna richiede stato, eventi, flussi complessi → al crescere della complessità, emergono i limiti di JS

La guerra dei «browser» (I)



- Innovazione veloce → il collo di bottiglia diventa JavaScript
- I browser introducono nuove feature a ritmo alto → frammentazione, comportamenti diversi, compatibilità
- Con app più ricche, il limite non è più solo HTML/CSS, ma anche le performance del motore JavaScript
- Si arriva ai motori moderni, soprattutto V8 (la base tecnica che rende possibile Node.js)



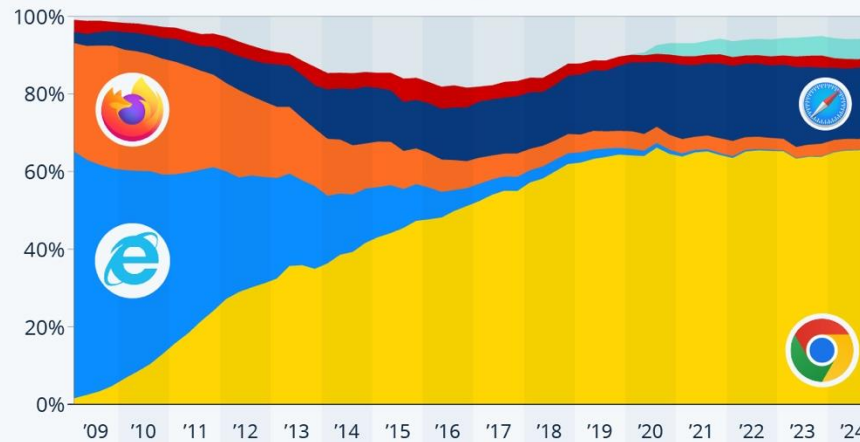
La guerra dei «browser» (II)



Chrome's Rise to Browser Dominance

Global market share of the most popular internet browsers*

Chrome Internet Explorer Firefox Safari Opera Edge Others



* All platforms except for gaming consoles

Source: StatCounter GlobalStats



statista

Il «motore» che esegue Javascript



- **Runtime** = ambiente che interpreta/compila ed esegue JavaScript
- Diverse versioni del «runtime engine» di JavaScript
 - **SpiderMonkey** (1996) – il motore storico, sviluppato Brendan Eich; ancora in uso in Mozilla Firefox
 - **Rhino** (1997) – uno dei maggiori motori JavaScript, distribuito con licenza open-source; scritto in Java e attualmente attivo e aggiornato
 - **V8** (2008) – Motore JavaScript open-source diventato famoso per le sue performance; sviluppato da Google
 - Elevate prestazioni: ottimizzazione, JIT (Just-In-Time compilation), inline caching
 - JS diventa più efficiente ➔ viene utilizzato per problemi più complessi (non solo scripting)

V8: la base che abilita Node.js



- **V8** rende JavaScript competitivo in performance → JS diventa interessante da utilizzare anche fuori dal browser
- Nasce **Node.js = V8 + runtime server-side**
 - Stesso linguaggio su *client* (UI/interazioni) e *server* (API/servizi)
 - Effetto ecosistema: JS non è più solo frontend
 - Abilita servizi web in JS con un ecosistema enorme (**npm**)
 - Modello event-driven + asincrono (ideale per carichi I/O intensivi)

Typescript: typing «su» JavaScript



- JavaScript «scala male» quando la complessità cresce → più team, più moduli, più refactor → più rischio di bug «silenziosi»
- **TypeScript** aggiunge un «sistema di tipi» che lavora a tempo di compilazione (in produzione, gira comunque JavaScript)
 - Controlli (errori prima di eseguire)
 - Autocomplete
 - Refactor più sicuri
- TypeScript è il «collante» dello stack moderno: React (UI) e Node (back-end)

NPM e la supply chain del JavaScript

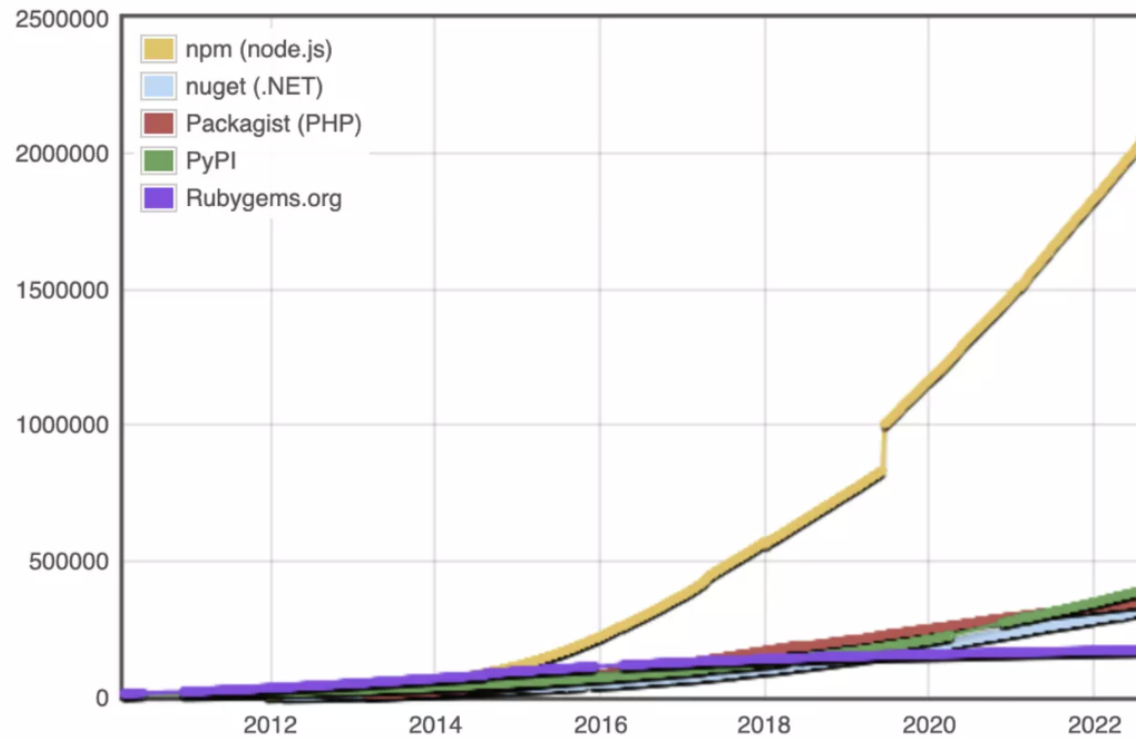


- Perché non basta un runtime?
 - Un runtime (**Node.js**) esegue codice
 - Ma per scalare serve distribuire e riusare codice → si rafforza l'importanza dei package manager
- **npm** diventa la base della supply chain JS
 - Effetto sviluppo rapidissimo
- Rovescio della medaglia: dipendenze molto profonde, vulnerabilità e aggiornamenti continui, pacchetti compromessi, *typosquatting*
- Alternative: yarn, pnpm

Ma quanto è diffuso NPM?



nmp «package count»





- ✓ Con **Node.js** non è difficile partire... ma nei progetti grandi cresce:
 - ✓ Complessità, coupling, «*codice spaghetti*»
 - ✓ Team multipli, refactor frequenti
- ✓ Cos'è **NestJS**?
 - ✓ Framework back-end per Node.js, scritto in Typescript
 - ✓ «*Opinionated*»: fornisce un'architettura standard
- ✓ Organizza il back-end in *modules* (importing e composizione), *controllers* (API/routing), *services* (logica di dominio)
- ✓ Integra facilmente REST, auth, validation, logging, ORM, microservizi

API moderne: REST come baseline



- UI moderne: bisogno continuo di dati
- **REST** si diffonde perché è semplice e standard
 - *risorse* (URI)
 - *metodi HTTP* (GET/POST/PUT/PATCH/DELETE)
 - *stateless*
 - *separazione client/server*
- Endpoint chiari e componibili
- Scalabilità orizzontale facilitata
- REST ≠ endpoint JSON → insieme di vincoli che guidano design, evoluzione e manutenzione
- TypeScript utile per definire esplicitamente i tipi di dati scambiati tramite le REST API – DTO (Data Transfer Object)



- UI ricche → problemi di fetch → query-driven API
- Limiti tipici di REST con UI ricche
 - *over-fetching*: download di più dati del necessario
 - *under-fetching*: servono più chiamate per comporre una vista
- La risposta: **GraphQL**
 - Linguaggio di query per API
 - Il client chiede esattamente i campi che servono
 - Meno *payload* inutili, meno *round-trip*
- Serve uno schema, *resolver* (da dove prendo il campo richiesto) e *caching* (cosa salvo in cache) più complessi
- Esempio di come l'evoluzione delle UI spinge l'evoluzione delle API
- Anche in questo caso, TypeScript aiuta a tipizzare

Type safety end-to-end: tRPC



- Idea chiave: **TypeScript** come «*contratto vivo*» tra backend e frontend → il tipo di backend guida automaticamente il tipo del client
 - Il client «inferisce» i tipi direttamente dal TypeScript del backend
- Velocità (meno ripetizioni) e sicurezza
 - Particolarmente efficace per team piccoli/medi e *monorepo* (backend e frontend nello stesso repo)
- Trade-off
 - Non sempre la scelta migliore (REST e GraphQL restano ottimi modelli)
 - Rappresenta un trend moderno: ridurre attrito e bug sfruttando TypeScript come lingua comune su Node + React

REST vs GraphQL vs tRPC



- **REST**

- Modello: *risorse + HTTP* (GET/POST/PUT/DELETE)
- Punti forti: standard, semplicità, ottimo per API pubbliche

- **GraphQL**

- Modello: *schema + query* (il client chiede i campi che servono)
- Punti forti: riduce over/under-fetching, aggrega dati da più servizi, comodo per UI complesse

- **tRPC**

- Modello: *RPC tipizzato end-to-end* (chiamate «tipo funzione»)
- Punti forti: typesafety totale con TS
- Per prodotti interni TS-first (React + Node), monorepo/single team, API non pubbliche

Front-end moderno: AJAX → SPA



- La svolta di **AJAX** (*Asynchronous JavaScript And XML*)
 - Chiamate asincrone dal browser
 - Aggiornamento di porzioni di UI senza ricaricare la pagina
- Cambia il paradigma
 - La pagina non è più un documento
 - È un'app che sincronizza UI ↔ dati ↔ eventi
- Nascono le **Single Page Applications (SPA)**
 - L'app gira nel browser e gestisce routing, stato, chiamate a API (REST/GraphQL)

React.js: la SPA «diventa ingegneria»



- Il prezzo delle SPA
 - State management (UI state + server state)
 - Performance
 - Bundle size (downloading, parsing)
 - Sincronizzazione dati client e server (caching, invalidation, accessi concorrenti)
- React come risposta
 - UI come funzione dello stato
 - Componenti riusabili + composizione
 - **Approccio dichiarativo**: descrivo cosa voglio vedere, non come manipolare il DOM
 - **Virtual DOM** → update model più gestibile

«Mostra subito» o «interagisci subito»



- **SSR (Server-Side Rendering)**: il server genera HTML pronto e lo invia al browser → il contenuto è subito visibile
- Cosa manca all'HTML SSR? La pagina si vede, ma non è ancora interattiva
- Il concetto di **«hydration»**: un processo in cui il client
 - scarica il bundle JS (o TS)
 - collega componenti + event handler all'HTML già renderizzato
 - ricostruisce lo stato necessario per rendere la pagina interattiva
- **Non un processo gratis**: CPU (parsing/esecuzione, binding eventi), memoria, tempo prima della piena interattività

Futuro: verso app «transitional»



- Un'app «**transitional**» combina più strategie di rendering nella stessa applicazione, scegliendole per pagina o per sezione
- Un unico modello non ottimizza tutto:
 - **SPA pura** → spesso molto JavaScript/TypeScript e avvio lento
 - **SSR ovunque** → costo server + hydration non banale
- Esempio concreto: e-commerce
 - Home/categorie: SSR
 - Prodotto: SSR (titolo, prezzo, descrizione,...) + zone interattive («**isole**») attivate con JS/TS solo dove serve (selezione taglia/colore/varianti, bottone Aggiungi al carrello, wishlist)
 - Admin: SPA

Programmazione vs progettazione



«Il web è un sistema informativo?»

- Eccessivo, perché l'intero web non è certo sotto il controllo di un singolo soggetto (neppure di più soggetti coordinati fra loro)

«Un sito è un sistema informativo e il web è una federazione di siti fra loro autonomi»

- Prospettiva più equilibrata, ciascun sito è sotto la responsabilità di un soggetto (o un gruppo di soggetti)
- Valido soprattutto per *«data-intensive web sites»*: grandi quantità di dati, significatività della struttura ipertestuale

Indipendenza dei dati negli ipertesti



Dati

- Quali informazioni vengono offerte attraverso il sito e quali sono i dettagli concettuali e l'organizzazione logica

Iperstesto

- Come sono organizzati i dati in pagine e quali link navigazionali collegano le diverse pagine

Presentazione

- L'aspetto di ogni informazione nelle pagine

Possiamo cambiare ipertesto e presentazione senza cambiare i dati



- All'aumentare delle attività produttive svolte attraverso il web, aumenta la possibilità di individuare regolarità nella struttura delle pagine
 - In un catalogo, è presumibile che le pagine relative ai vari prodotti abbiano una struttura simile (se non uguale)
- Pertanto, può avere senso applicare i concetti di modello e di schema anche nel contesto degli ipertesti e del web

Dati

Ipertesto

Presentazione

ER, relazionale, UML

???

HTML/CSS, XML

Quale modello per gli ipertesti?

Quale metodologia di progettazione che attraversi tutti i livelli?



Progettazione di applicazioni web full stack

Introduzione al corso



Prof. Devis Bianchini
Dip. di Università degli Studi di Brescia